An implementation of Open-RMF in an industrial environment

Michele Belletti

May 7, 2025

Contents

Part I

Introduction

This work presents a study on the new system, OPEN-RMF, which is still under active development but has reached a mature enough state to be deployed with robots at the Elettra Sincotrone Trieste facility.

Chapter 1

Background and motivation

Elettra Sincontrone Trieste[1] s a major research facility that primarily provides users access to two light sources: the synchrotron Elettra and the free electron laser Fermi. In addition to operating these sources, the facility actively researches new technologies and techniques to enhance its capabilities and operations. Many recent advancements are based on robotics, which, especially after the COVID-19 pandemic and with the rapid rise of large language models (LLMs), have enabled new ways for users to interact with the infrastructure, including robot-assisted information retrieval and package delivery services. Previously, the facility developed a global planner, D-Star (D-star based optimized trajectory planner)[17], within the ROS 1 framework to enable robot navigation across large-scale environments. However, as ROS 1 has reached end-of-life, the codebase must now be migrated to ROS 2. Additionally, the facility employs robots from multiple vendors, each specialized in different tasks and not designed to coordinate with one another. In an environment that shares resources such as narrow passages and lifts, this lack of interoperability creates significant coordination challenges. To address this issue, Open-RMF[2] has been developed in recent years as a multiplatform system for controlling and managing heterogeneous fleets of robots.

[1]https://www.elettra.eu/
[2]https://www.open-rmf.org/

Chapter 2

Research questions and objectives

The objectives of this thesis are as follows: to integrate a Rover Mini robot from the company RoverRobotics [1] into the ROS 2 Nav2 ecosystem, to port the D-Star global planner from ROS 1 to ROS 2,

Figure 2.1: RoverRobotics Rover Mini

incorporate the robot into the Open-RMF framework for centralized fleet management and evaluate the suitability of the Open-RMF ecosystem for deployment at the Elettra Sincrotrone Trieste facility.

[1]https://roverrobotics.com/

Chapter 3

Scope and limitations

The principal limitation is the current lack of access to lift control systems, which prevents direct integration and operation by the robot fleet. Additionally, there is no automated system for opening and closing doors. While Open-RMF offers a high degree of customizability, and human-in-the-loop interactions can be configured—such as triggering a light or sending a notification when a robot approaches a door—this method requires a human to manually open the door or operate the lift. However, such an approach is not ideal for achieving full autonomy.

Part II

Literature Review

In this section, we provide a description of the key technologies used in this thesis. We begin by describing the Robot Operating System (ROS) and its functionality, followed by an overview of the navigation system employed, and a discussion of how the Open-RMF fleet management system operates.

Chapter 4

Overview of ROS1 and ROS2

The Robotics Operating System (ROS)[1] is an open-source collection of tools and libraries designed to support the development of robotics applications. Despite its name, ROS is not an operating system (OS) in the traditional sense, but rather a software development kit (SDK) that provides developers with a comprehensive set of tools for building robotic systems. It offers several features typically associated with an OS, including hardware abstraction, low-level device control, implementation of commonly used functionalities, inter-process communication, and package management. The ROS ecosystem is built upon the following core components:

- **Middleware communication layer (RMW)**: Built on the Data Distribution Service (DDS) standard, this layer enables nodes to exchange information using an anonymous publish/subscribe pattern. This design promotes fault isolation, separation of concerns, and modular interfaces. ROS also supports synchronous, Remote Procedure Call (RPC)-style communication between services. While the default DDS implementation is eProsima Fast DDS (`rmw_fastrtps_cpp`), this work uses Eclipse Cyclone DDS (`rmw_cyclonedds_cpp`) due to its better compatibility with the Nav2 navigation stack.

- **Development tools**: ROS provides a suite of tools for managing and enhancing software development, including utilities for launching, introspection, debugging, visualization, plotting, logging, and playback.

- **Multi-machine support** ROS includes tools and libraries for retrieving, building, writing, and executing code across multiple computers and OS.

- **Community and governance**: ROS is supported by an active and extensive global community, coordinated primarily by Open Robotics[2], which serves as the central organization for its development and maintenance.

[1] https://www.ros.org/
[2] https://www.openrobotics.org/

4.1 Structure of ROS

The official ROS documentation[3] provides comprehensive information about the system. All communication in ROS is handled through the **ROS Graph** , a peer-to-peer network of processes (called nodes) that execute concurrently and exchange information with one another. Some components of this graph are described below. Note that some of the functionalities described are specific to ROS 2 and are not available in ROS 1:

- **Nodes**: A node is a modular process that performs a specific task. Nodes communicate with each other through topics, services, actions, and parameters. In ROS 2, nodes can be implemented as **LifecycleNode**, which provide a managed state machine with defined transitions for startup and shutdown, enabling more deterministic behavior.

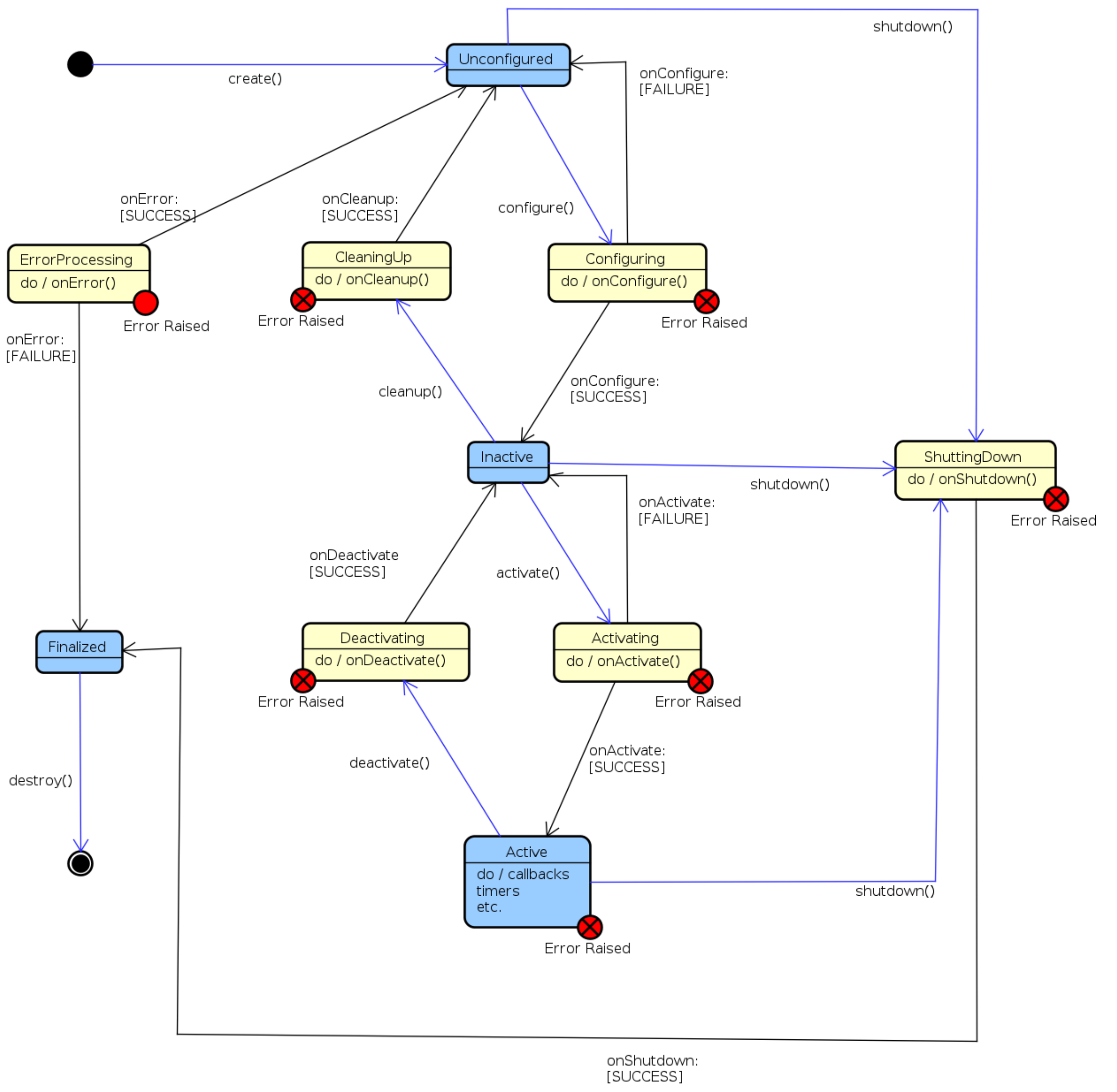

Figure 4.1: State Machine of LifecycleNode

Nodes can also be composed within the same process using the **Composition** feature, which improves performance by bypassing middleware overhead[6].

[3]https://docs.ros.org/en/rolling/index.html

- **Topics**: Topics are used for communication between nodes to share streams of messages in a publisher-subscriber structure.

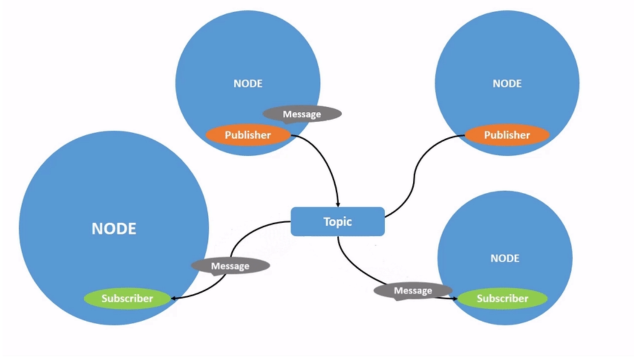

Figure 4.2: Topic process

- **Service**: Services are based on the request-response model. When a node with a service client sends a Request Message to a node with a service server, it receives a Response Message from that specific server.

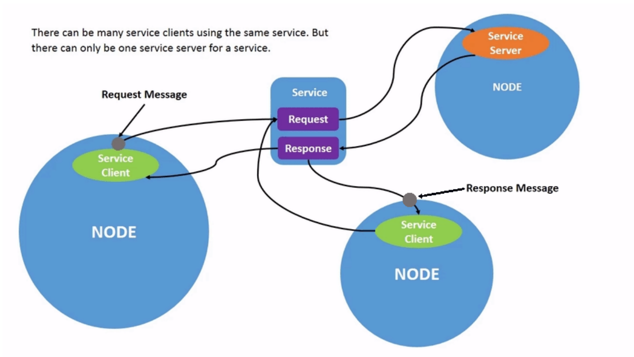

Figure 4.3: Service process

- **Parameters**: Parameters are configurations of a node that can be modified without changing the source code.

- **Actions**: Actions are designed to support long-running tasks that require feedback and the capability to interrupt when needed. They combine two services (goal and result) and a publisher-subscriber mechanism (feedback). A node sends a goal request and receives an acknowledgment, then starts publishing feedback messages while executing the task, and finally responds with the result.

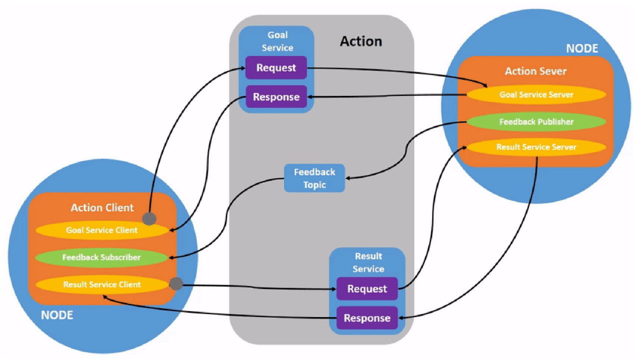

Figure 4.4: Action process

4.2 Tools of ROS

Now, I will describe the tools that are built into ROS for developing robotic systems:

- **Launch**: A file structure that allows the simultaneous start of multiple nodes with a set of parameters.
- **rqt**: A graphical user interface (GUI) tool for ROS. It includes a series of plugins that facilitate development, monitoring, and diagnostics of the ROS system.
- **rqt_console** A plugin for rqt that visualizes the logs generated by the ROS.
- **Rviz**: A 3D visualization tool for the Robot Operating System. It allows visualization of sensor data and state information from ROS.

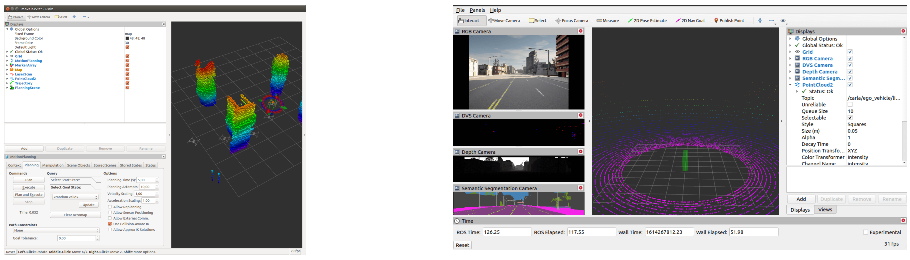

Figure 4.6: Examples of visualization of ROS data with Rviz

- **rosdep**: A command-line utility for installing system dependencies required by ROS packages.
- **sros2**: A security extension for ROS 2 that provides tools and instructions to apply security policies to DDS (Data Distribution Service).

Another important category of tools that are integrated with ROS is simulation environments. These environments allow developers to test their code on simulated robots under various conditions, which helps identify bugs that could potentially harm a real robot or its surroundings. Simulation tools provide realistic physical behavior for all components of a robotic system. According to the ROS documentation, two main simulation tools are commonly used: Gazebo and Webots. Among them, Gazebo is also maintained by Open Robotics and is widely adopted in many projects, including the one discussed in this paper.
Gazebo offer:

- **ROS Integration**: A bridge that converts Protobuf messages to ROS messages, enabling communication between Gazebo and ROS.
- **Performance tools**: Features for distributed simulation across servers, dynamic asset loading, and adjustable time steps to optimize simulation performance.

- **Realistic simulation**: Support for a variety of sensors, including their noise models, within a 3D world rendered using OGRE 2.1[4] and powered by multiple physics engine, the default one is DART[5].

- **Extensibility**: A modular design that supports plugins, allowing developers to run custom code that interacts directly with the simulation environment.

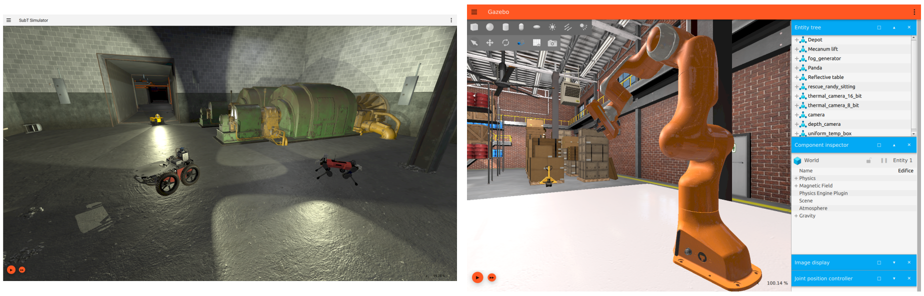

Figure 4.8: Examples of Gazebo

[4]https://wiki.ogre3d.org/Home
[5]https://dartsim.github.io/

Chapter 5

Introduction to NAV2 and Google Cartographer

Multiple tools are available to enable a robot to move autonomously within an environment. One of the most complete systems is Nav2 [1] [9], the successor to the ROS 1 Navigation Stack.

5.1 Description of Nav2

Nav2 provides a comprehensive set of tools for perception, planning[12], control, localization, and visualization, allowing any robot to navigate autonomously and reliably, even in complex environments. It enables building a model of the environment from sensor data, dynamically calculating the optimal path to avoid obstacles, setting motor velocities, and executing navigation tasks using Behavior Trees.

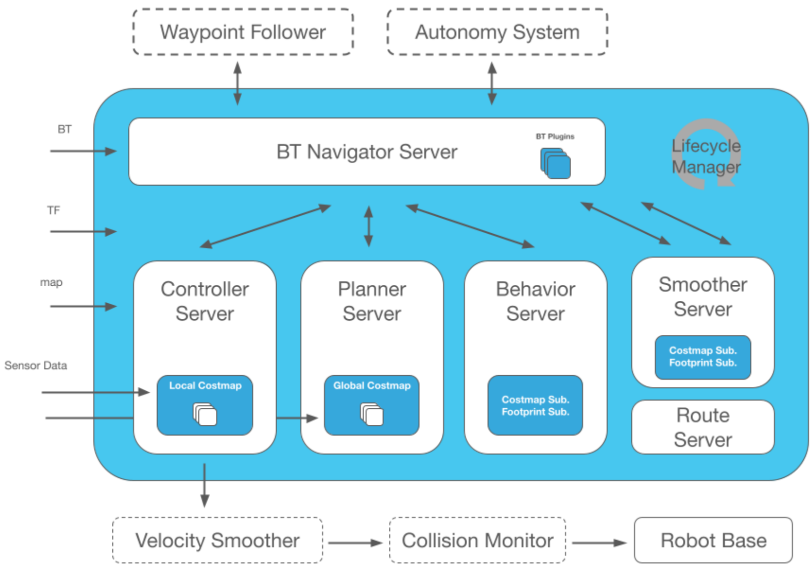

Figure 5.1: Nav2 architecture

[1] https://nav2.org/

Nav2 introduces several improvements over ROS 1, including:

- **Behavior trees**[2]: Nav2 uses Behavior Trees to structure and execute tasks in a human-readable and modular way. These trees enable the creation of complex systems and can be tested with various tools. The BehaviorTree.CPP[2] library is used to implement them, and it allows linking different trees for flexible behavior management.

- **nav2_util LifecycleNode**: This is a wrapper around the standard `rclcpp_lifecycle::LifecycleNode` described earlier 4.1. It adds a bond connection to the Lifecycle Manager, which monitors the state of each node and can safely transition nodes through their lifecycle states in the event of a failure.

- **Action Server**: Described in 4.1, Nav2 uses action servers for Behavior Tree interactions, such as NavigationToPose (navigate to a specific location) and NavigationThroughPoses (navigate through multiple waypoints).

Nav2 is composed of several key nodes that work together to control the robot:

- **Controller Server**: Handles movement requests. It uses plugin-based modules such as a progress checker (to verify if the robot is making progress), goal checker (to determine if the robot has reached its destination), and a controller (to generate velocity commands using the **local costmap** while avoiding collisions).

- **Planner Server**: Computes the global path to the destination using sensor data and a plugin-based global planner loaded with the **global costmap**.

- **BT Navigator Server**: Implements the NavigateToPose, NavigateThroughPoses, and other task interfaces. It is a Behavior Tree-based implementation of navigation that is intended to allow for flexibility in the navigation task and provide a way to easily specify complex robot behaviors.

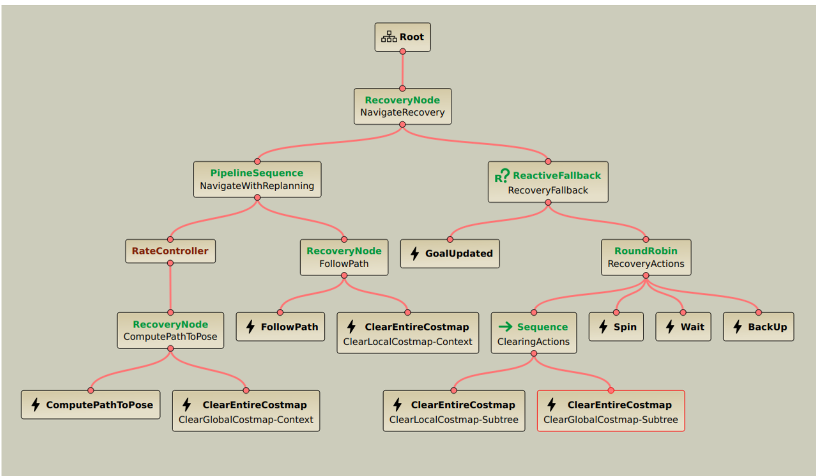

Figure 5.2: Humble Nav2 BT navigation `navigate_to_pose_w_replanning_and_recovery.xml` loaded with Groot2

[2]https://www.behaviortree.dev/

- **Smoother Server**: Smooth the path minimizing sharp turns and rapid rotations.

- **Behavior Server**: Handles specific behavior actions like spinning, backing up, and waiting.

- **Waypoint Follower**: An example of orchestrated control (an **Autonomy System**), this node takes a list of waypoints and uses the NavigateToPose action server to move between them, optionally performing custom tasks (e.g., taking a photo) at each waypoint through plugins.

- **Route Server**: Calculates routes through a predefined navigation graph instead of using free-space planning.

- **Velocity Smoother**: Smooths velocity, acceleration, and deadband signals sent to the robot to ensure safe and smooth motion.

- **Costmap** is a representation of the environment using a regular 2D grid of cells with a cost (unknown, free, occupied, or inflated) and is used for generate the global plan or to compute local control efforts.

The next things that Nav2 relies on for controlling the navigation are:

- **state estimation** of the robot that generate a suitable TF tree, a tree structure of coordinate frames generated using the transform library[3], based on the REP 105, that is should provide this structure: `map` → `odom` → `base_link`. According to the REP 105 they are defines as:

  - `map` : A world-fixed frame that may change in discrete jumps. It's useful for long-term global reference but not suitable for short-term localization.
  - `odom` : A continuous world-fixed frame that may drift over time. It provides short-term local reference.
  - `base_link`: A robot fix frame.

  The `odom` → `base_link` link can be estimated using various sensors. For improved accuracy, we use the `robot_localization` package[11], which provides nonlinear state estimation using Kalman Filter fusing the robot's odometry and the IMU data.
  The `map` → `odom` link can be generate with a global positioning system like Simultaneous Localization And Mapping (SLAM) method or using the one provided by Nav2, Adaptive Monte-Carlo Localization (AMCL).

- **map** can either be preloaded or built dynamically using a SLAM method. One SLAM method systems is Google Cartographer, which will be described in the next section. It is also the SLAM method that was used to generate the map for navigation.

5.1.1 Cartographer

Cartographer[4] is a SLAM method that operates using two main subsystems: a **Local SLAM** (called **Frontend**) that build a series of locally consistent **submaps** from successive scans and from the pose extrapolator with the possibility that it can drift during acquisition and a **Global SLAM** (called **Backend**) that reduces drift by performing loop closure—detecting when the robot has returned to a previously visited location. It uses scan matching against submaps and applies global optimization to align all submaps consistently and build a globally accurate map. Cartographer is capable of achieving mapping accuracy of 5 cm, but it requires extensive parameter tuning to reach optimal performance.

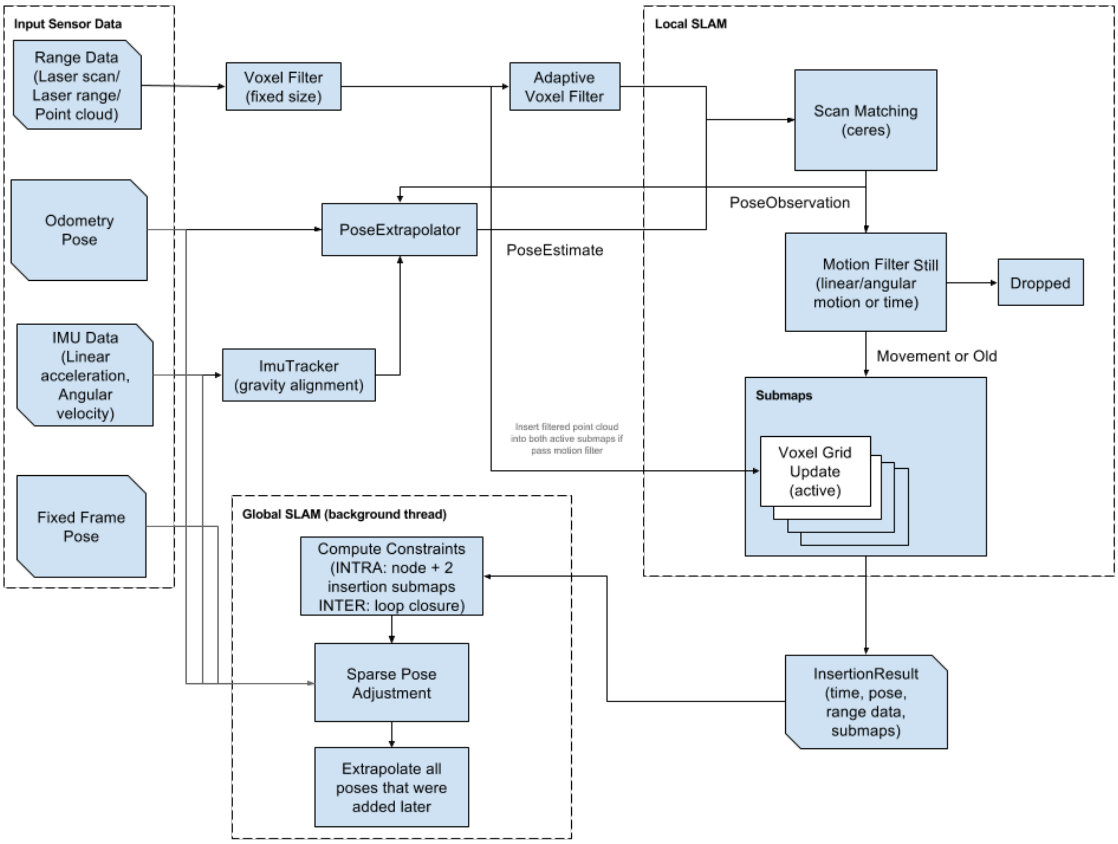

Figure 5.3: Cartographer Algorithm

Chapter 6

Review of related work on porting navigation stack from ROS 1 to ROS 2

The widespread adoption of the Robot Operating System (ROS) has introduced new use cases that were not originally considered during its development. Initially created as an academic project, ROS encountered limitations as users began applying it to more complex scenarios such as fleet management, deployment on embedded boards and microcontrollers, real-time systems, unreliable networks, and industrial applications. These emerging requirements highlighted architectural constraints in ROS 1, ultimately necessitating a complete redesign, which led to the development of ROS 2. Comprehensive documentation for ROS 2 is available at https://design.ros2.org/ . Many companies have adopted the ROS 2 ecosystem, noting improvements in development workflows[8] and in specific subsystems like in Nav2[12]. New initiatives, such as micro-ROS[1], aim to bring ROS 2 capabilities to microcontrollers. Additionally, features like Node Composition offer improved performance optimization[6] while sros2 allow for a better security[14]. As outlined in these references, ROS 2 offers substantial advantages over ROS 1, making it the preferred framework for modern robotic applications. With this in mind, we will migrate the global planner using the official ROS 2 documentation[2] and various secondary sources [3].

[1]https://micro.ros.org/

[2]https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html

[3]https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

Chapter 7

Overview of open source fleet management systems, particularly Open-RMF

Currently, several fleet management systems are available:

- **Isaac Mission Dispatch**[1]: A microservice-enabled fleet management system developed by NVIDIA that allows users to submit missions to multiple robots and monitor both robot and mission statuses. It provides connectivity between robots and the fleet management system but does not handle logistics such as task allocation or conflict resolution (e.g., intersecting robot paths). The implementation relies on the VDA5050 protocol, an industry standard for communication between cloud control services and mobile robots, and uses MQTT, a lightweight, publish-subscribe network protocol designed for devices with limited resources and bandwidth.

- **ROOSTER Fleet Management**[2]:An open-source ROS 1-based project that supports heterogeneous fleet management with task allocation, scheduling, and routing functionalities.

- **Open Robotics Middleware Framework (Open-RMF)**[3]: A free, open-source, and modular system that facilitates interoperability between multiple robot fleets and physical infrastructure such as doors, elevators, and building management systems.

- **Robofleet**[4][13]: An open-source system based on ROS 1 that supports inter-robot communication, remote monitoring, and remote tasking for heterogeneous fleets of ROS-enabled service robots. It is specifically designed to handle network variability and to incorporate security control features.

Other frameworks provide the infrastructure needed to initiate fleet control:

- **Transitive Robotics**[5]: An open-source framework for full-stack robotics, designed to simplify the development of capabilities that connect robots, cloud

[1]https://github.com/NVIDIA-ISAAC/isaac_mission_dispatch
[2]https://github.com/ROOSTER-fleet-management
[3]https://www.open-rmf.org/
[4]https://github.com/ut-amrl/robofleet
[5]https://transitiverobotics.com/

services, and web front-ends through a dynamically updating shared data model. It addresses common challenges in building full-stack robotic applications, such as unreliable networks, intermittently offline robots, and cross-device versioning.

- **Robot Web Tools**[6]: A collection of tools that enable communication with remote robots via web technologies, facilitating remote monitoring and interaction.

Given that Open-RMF offers control over multiple fleets, enables structured interaction between them to achieve optimal outcomes, provides a comprehensive toolset for system development, is based on ROS 2, and is open source, it will be used as the foundation for our fleet management architecture.

7.1 Open-RMF description

Open-RMF is a flexible and modular toolkit built on ROS 2 that facilitates interaction between heterogeneous robot fleets. It employs standardized protocols to enable communication with infrastructure, environments, and automation systems, optimizing the utilization of critical shared resources such as elevators, doors, narrow corridors, and other robots. By coordinating access to these resources, Open-RMF introduces system-level intelligence that helps prevent conflicts and ensures proper task and resource allocation. Figure 7.1 illustrates the architecture and operational workflow of Open-RMF.

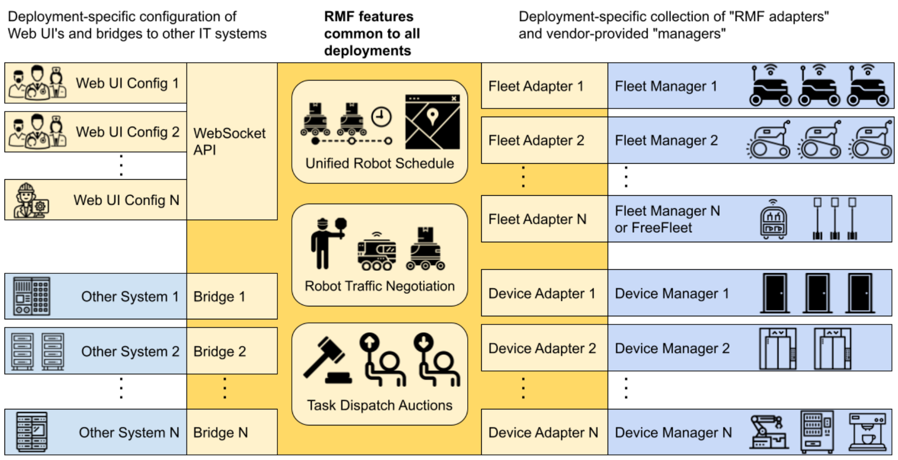

Figure 7.1: Open-RMF Structure

The fleets that can be controlled through Open-RMF can be categorized into three types:

- *Full Control*: Fleets of this type provide full status updates and allow complete control over path planning and execution.

- *Traffic Light*: These fleets also provide full status updates, but only allow limited control, such as pausing and resuming operations. Open-RMF has control over certain behaviors, but not over the specific paths.

[6]https://robotwebtools.github.io/

- *Read Only*: These fleets only provide status updates and cannot be controlled. Only one read-only fleet can exist in the system at any given time, as the presence of multiple read-only fleets would make it nearly impossible to avoid deadlocks or resource conflicts.

If a fleet lacks has *No Interface*, it can lead to deadlocks in resource allocation and is incompatible with an RMF system.
The core of Open-RMF, known as `rmf_core`, is composed of several modular packages, each serving a distinct role within the system:

- `rmf_traffic`: Provides core scheduling and traffic management capabilities. It is based on a platform-agnostic **Traffic Schedule Database** and includes two main mechanisms for traffic deconfliction:

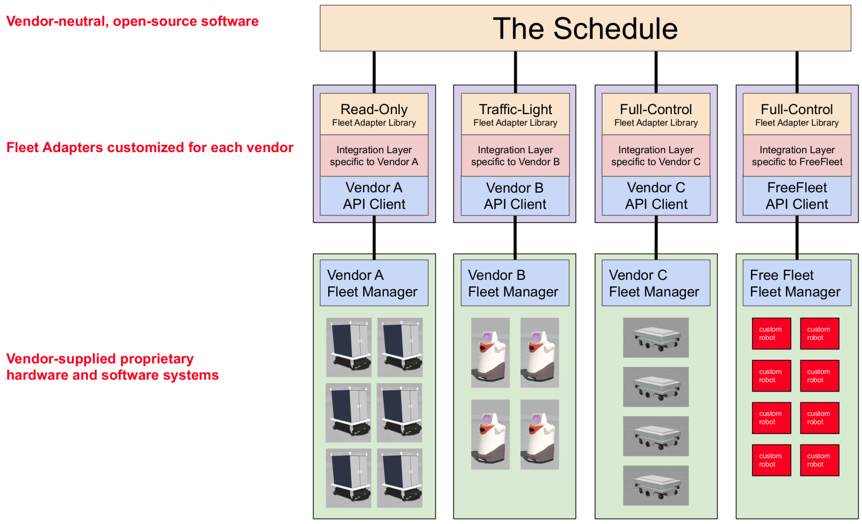

Figure 7.2: Open-RMF Schedule structure

  - *Prevention*: When changes occur in the database (e.g., route cancellations), fleet managers can proactively replan routes to avoid conflicts.
  - *Negotiation*: When a potential conflict is detected between two or more schedule participants, a conflict notice is issued to the relevant fleet managers. Each manager responds with a preferred path that accommodates other agents, and an external arbitration module selects the optimal path.

  Conflict avoidance for Automated Guided Vehicles (AGVs) is implemented using a time-dependent extension to A* search. This algorithm considers both spatial and temporal dimensions, allowing it to generate conflict-free paths by accounting for the future movements of other agents. Support for Autonomous Mobile Robots (AMRs) is under active development.

- `rmf_traffic_ros2`: Provides the necessary ROS 2 interfaces for integrating rmf_traffic into ROS 2-based systems.

- `rmf_task`: Contains APIs and base classes for defining and managing tasks in RMF. The framework natively supports three types of task requests:

– *Clean*: For robots capable of cleaning floor spaces.
– *Delivery*: For robots tasked with transporting items between locations.
– *Loop*: For robots required to repeatedly navigate between two or more points.

- `rmf_dispatcher_node`: Contains the logic responsible for task allocation and management across multi-fleet systems. When a user submits a new task request, the dispatcher intelligently assigns it to the most suitable robot by querying each fleet adapter capable of performing the task. These adapters respond with bids indicating which robot is best suited, and RMF uses this bidding mechanism to determine the optimal assignment, as illustrated in Figure 7.3.

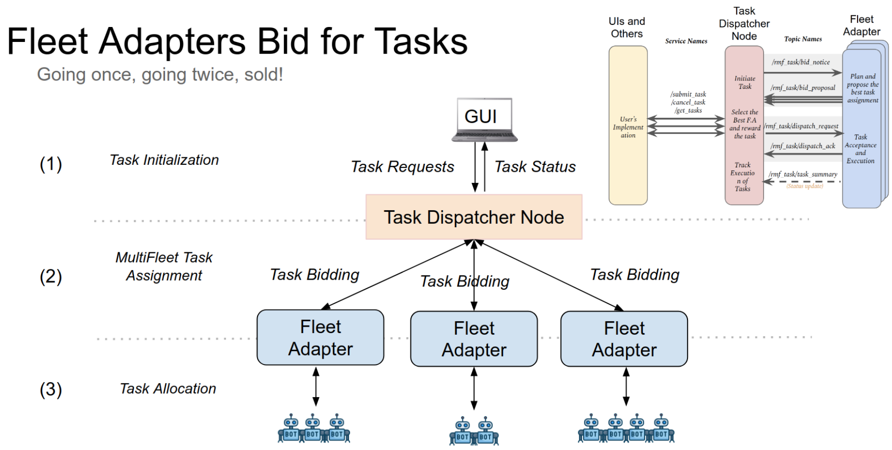

Figure 7.3: Open-RMF Bid Task

- `rmf_battery`: Provides battery consumption estimation models for different robot activities, aiding in energy-aware task planning.
- `rmf_ros2`: Supplies ROS 2 adapters, nodes, and Python bindings to integrate the RMF core components into ROS 2 applications.
- `rmf_utils`: A collection of utility functions and tools used across various RMF packages.

The following tools are available to support development with Open-RMF:

- **RMF demos**: A collection of demonstration packages showcasing the capabilities of Open-RMF. These serve as a useful starting point for development and experimentation.
- **Traffic Editor**: A graphical map editor used to define traffic patterns and interaction points (e.g., doors, lifts) that RMF uses for traffic management. It also supports the generation of Gazebo simulation worlds based on the designed maps.

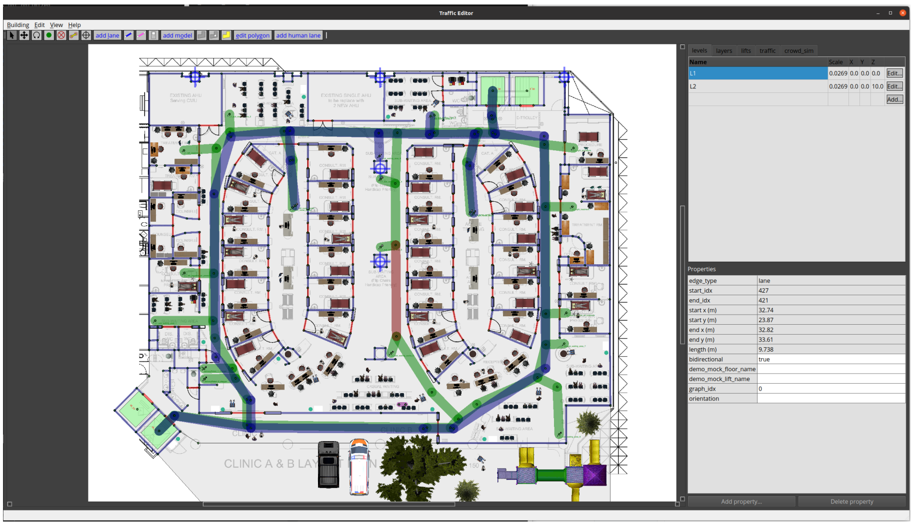

Figure 7.4: Traffic Editor

- **Free Fleet**: An open-source fleet adapter that enables the integration of standalone mobile robots into Open-RMF, allowing the creation of heterogeneous fleets.

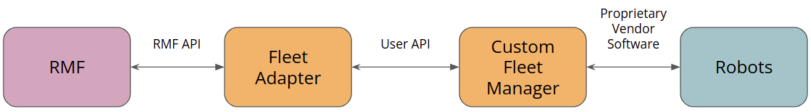

Figure 7.5: Fleet Adapter

- **RMF Schedule Visualizer**: An RViz plugin that provides visualization and monitoring of the RMF schedule, enabling users to observe and interact with traffic and task execution in real time.

- **RMF Web UI** : A web-based application for visualization and control of the Robotic Middleware Framework (`rmf_core`). It provides a user-friendly interface for monitoring system state and interacting with fleet operations.

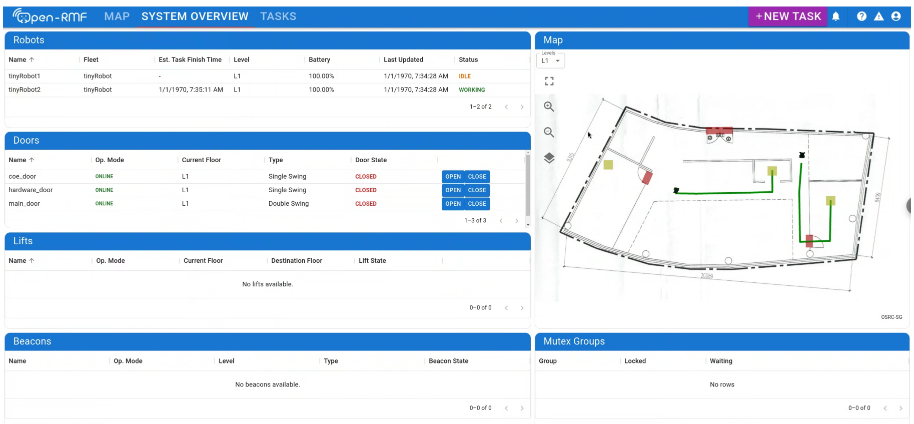

Figure 7.6: RMF Web

- **RMF Simulation**: A set of Gazebo plugins designed to simulate various aspects of building infrastructure—such as doors, lifts, and crowds—as well as robot behavior, including navigation and interactions with workcells.
- **Simulation assets**: Freely available 3D models and environment assets used to support simulation development and testing in Gazebo.

Part III

Methodology

In this part we will discuss all the procedure that took place from building our testing platform, based on the Rover Mini and the sensors used. Setting up the Nav2 navigation stack. The procedure used to port the Global planner from ROS 1 to ROS 2. How we integrate the system and the software that is used to setup the Open-RMF system. All the code is available to this github repo `michbelle/myTesiCode` [7] and allow also the simulation of our platform. For the simulation is going to be used Docker, that allow to have a base container that not depend of the OS of the machine. The Docker image is based on `osrf/ros:jazzy-desktop-full` and install all the code used in this thesis and also all the open-RMF tools and demos. Tested were done with a i5-7600K CPU and a GeForce GTX 1060 6GB on Ubuntu 22.04, the software was laggy so it reccomend to use more recent hardware.

[7] https://github.com/michbelle/myTesiCode

Chapter 8

Description of research platform and hardware used

The robot is a 4 wheel drive (WD) skid-steering (see Figure 2.1)that has a maximum speed of 12.87 km/h with a payload of 45,36 kg. With an operating time of Operating Time 60-90 minutes active or max 6 hours in idle. The company offer the code to control it and also to simulate on Gazebo on github[1]. The code is steel tested only for ROS 2 Humble, this is going to be our version of ROS 2 that we are going to use for navigation on the Rover.
For controlling it, the rover give a USB terminal for connecting to a PC or single board computer. For this purpose we are using a Mini PC where there is all the control of all the navigation based on Nav2. The Mini PC is a Beelink SER3-E-16512EJ0W64PRO with a Ryzen 7 3750H Picasso processor which is a quad-core 8-thread 2.3 GHz mobile processor boosting to 4.0 GHz with Radeon RX Vega 10 Graphics. The power that it's necessary from it to work will be took from the Rover that can provide 19V.

Figure 8.1: Mini PC used

On top of that to increase the estimation of odometry we will mount an IMU Witmotion (WIT) JY901B. A 9-Axis Combined IMU/Magnetometer/Altimeter (linear accelerations, angular velocities, Euler angles, magnetic field, barometry, altitude). The data from the sensor use a TTL/UART protocol that are collected using the libeary Witmotion Sensors UART Connection Library[16],using a channel bridge chip USB to serial, and then published into ROS 2 ecosystem through a node

[1] https://github.com/RoverRobotics/roverrobotics_ros2

witmotion_ros using the code from this repo in github ElettraSciComp/witmotion_IMU_ros[2].

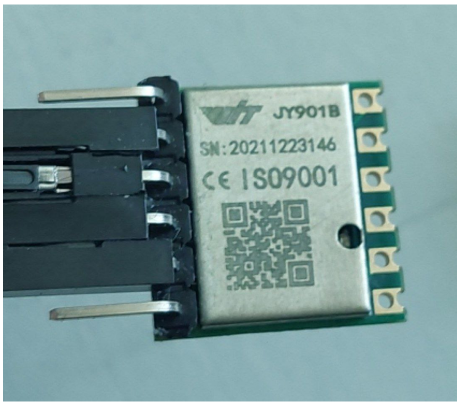

Figure 8.2: IMU used

The last sensor necessary to complete the navigation that is used in localization as described in 5.1 to generate a link between the TF frame `map` and `odom` that compensate the errors generated by the model and to see obstacles is a Laser 2D scan because gives as information in all the directions using only a sensor and we built have built only a 2D map, respect at other tools are available [1][10] that may require more computer power. The Laser scan used is RPLIDAR S1, 360 degree 2D laser scanner (LIDAR that is light detection and ranging) with a distance range of 40m on white object and 10m on black ones, a sample rate of 9.2kHz, scan rate from 8 to 15 Hz, so an angular resolution of 0.313-0.587°, accuracy of ±5cm and resolution of 3cm. The code is available on github Slamtec/sllidar_ros2 [3].

Figure 8.3: LIDAR used

[2] https://github.com/ElettraSciComp/witmotion_IMU_ros/tree/ros2
[3] https://github.com/Slamtec/sllidar_ros2

Then all is mounted on the Rover as in the following picture, where also modeled and printed with a 3d print a support for each sensor.

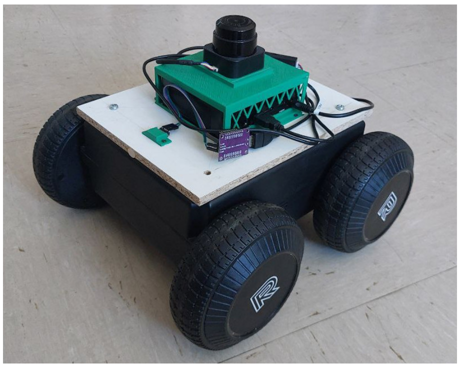

Figure 8.4: Robot used

Then we edit the URDF (Unified Robot Description Format), a file format for specifying the geometry and organization of robots in ROS, given from the producer to better fit the new configuration to allow the nodes that are going to use the sensor to know the exact position and orientation of the sensors respect the `base_link`.

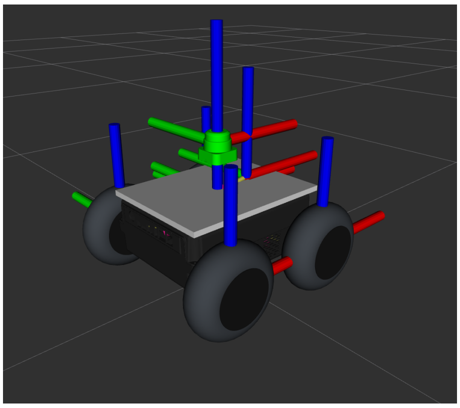

Figure 8.5: 3D model with TF

This allow to launch the following nodes:

- `0.1r_sensorLaunch.launch.py`: this file allow to launch the nodes that publish the data of the IMU and LIDAR.
- `0.2r_mini.launch.py`: this publich the data of the odometry given by the encoder data built in the rover.

For simulation:

- `0s_ign_mini_gazebo.launch.py`: launch the simulation in gazebo of the robot and using the node `ros_gz_bridge/parameter_bridge` share the message generate in gazebo with ROS 2.

Chapter 9

Steps taken to port the navigation stack from ROS 1 to ROS 2

In this capter we will split the parts in function of the different part that we are going to use of nav2 with particular attention of the porting of the code of the global planner from ROS 1 to ROS 2.

9.1 odometry estimation odom $\rightarrow$ base_link

As described before the odometry data given from the rover arent good and the navigation stack report big errors in rotation a well know fact of problem in estimation using skid-steering vehicles. For solve this problem we use the package robot_localization in specific the Node ekf_node. This package allow fusing multiple sensor also of the same type, allow to publish in multiple message type like nav_msgs/Odometry, allow to edit message from each sensor before insering in the estimation and is a continuous estimation that allow to estimate the position even if data is going to missing for a long period. For launching the node there are provide the following launch file:

1<r/s>_robot_localizationEKF.launch.py

9.1.1 odometry estimation configuration

Has describe before and also in the doc[1] of the node for adding the sensors we added this rows at the configuration file odom0:odometry/wheels and imu0:imu/data that are the estimation described before. Also we need to choose which components of the state we want to fuse using this array and set true if we want it to fuse:
sensorX_config:$[X, Y, Z, roll, pitch, yaw, \dot{X}, \dot{Y}, \dot{Z}, \dot{roll}, \dot{pitch}, \dot{yaw}, \ddot{X}, \ddot{Y}, \ddot{Z}]$
for the IMU we used only the yaw velocity while for the odometry we select only the velocity along the x and y.
Then is also possible to tune the process noise covariance matrix Q and represents the noise we add to the total error after each prediction step. In general, the larger the value for Q relative to the variance for a given variable in an input message, the faster the filter will converge to the value in the measurement.

[1] https://docs.ros.org/en/noetic/api/robot_localization/html/index.html

9.2 localization `map` → `odom`

In this part we will describe how archive the link frame from `map` to `odom`. As described before the costmap was create with Cartographer, for testing both SLAM from `slam_toolbox`[5] (github: SteveMacenski/slam_toolbox[2]) and `cartographer_ros`[4] (github: ros2/cartographer_ros[3]) package were used in this thesis.
With cartographer we obtain this map:

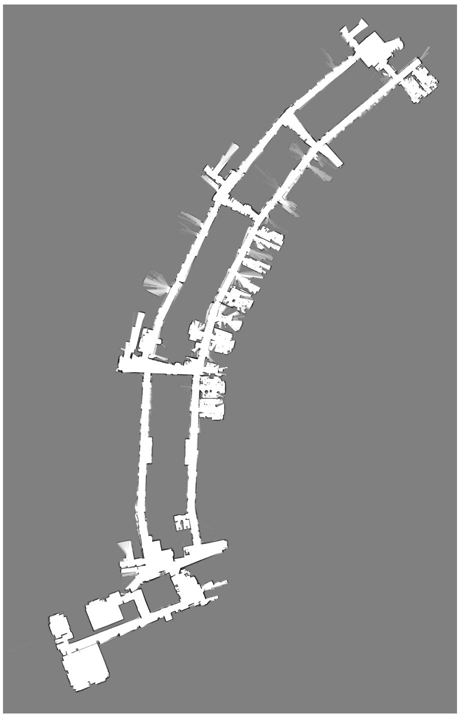

Figure 9.1: Map acquired with Cartographer

Then it was tried the use of cartographer as only for localization but because the project is not more maintained was not found possible to use it correctly, so the solution was to use the one given from nav2 package `nav2_amcl` and node `amcl`.
But this has the problem that we need to convert the map acquired with cartographer as a `.pbstream` to a pair of `.pgm` and `.yaml` for the new Node. This was done using this command:

```
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
        -pbstream_filename map.pbstream \
        -map_filename map.pgm \
        -resolution 0.05
```

This bringht some minors errors to the map thiking there was some place capable to move on, repaired usign gimp and optainint this map (also correct error to allow from my workstation to not have a obstacle not present)

[2] https://github.com/SteveMacenski/slam_toolbox/tree/ros2
[3] https://github.com/cartographer-project/cartographer_ros

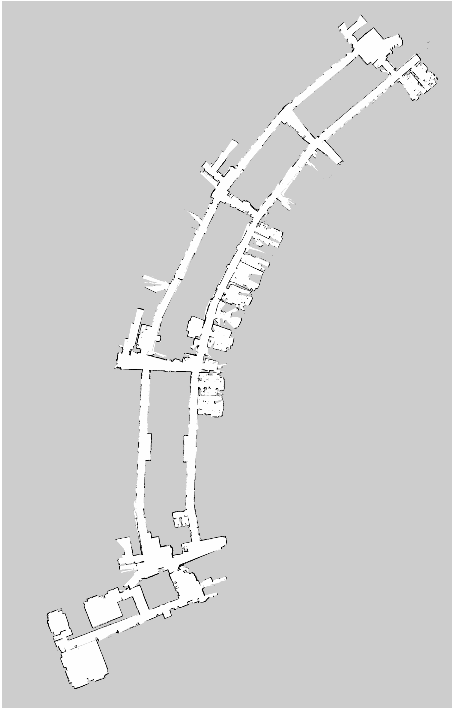

Figure 9.2: Map edited with Gimp and used

Then it was possible to launch the localization using:
3<r/s>_L<amcl/Cartographer>.launch.py
While for mapping are provided the code:
X<r/s>_mappingW<slam/cartographer>.launch.py

9.3 navigation stack

In this section we will discuss what structure for the navigation stack is used and how we port the Global planner from ROS 1 to ROS 2.

9.3.1 navigation launch file

In this file we will use the new structures of ROS 2, the composition node and the lifecycle manager. The file was written to use the ability of composition if it is enable in the launch configuration so it can be operate in boths ways. The file is launching all the following packages with node or composition plugins:

- package `nav2_controller` with Node `controller_server` or plugin `nav2_controller::ControllerServer`
- package `nav2_smoother` with Node `smoother_server` or plugin `nav2_smoother::SmootherServer`
- `nav2_planner` with Node `planner_server` or plugin `nav2_planner::PlannerServer`
- `nav2_behaviors` with Node `behavior_server` or plugin `nav2_behaviors::BehaviorServer`

- nav2_bt_navigator with Node bt_navigator or plugin nav2_bt_navigator::BtNavigator

- nav2_waypoint_follower with Node waypoint_follower or plugin nav2_waypoint_follower::WaypointFollower

- nav2_lifecycle_manager with Node lifecycle_manager or plugin nav2_lifecycle_manager::LifecycleManager

The parameters where not as much changed by the default configuration, the only change we need to apply is in the planner_server to have our navigation system after porting it to ROS 2. In the next section is the step we took to port it to ROS 2.

9.3.2 Porting DStarGlobalPlanner to ROS 2

The DStarGlobalPlanner is a implementation of the D* informed incremental search algoritm with external optimizers[15] for the ROS 1 move_base package[4]. In this part we will refer a lot on the porting documentation refered before [5][6] and the documentection from Nav2 : "Writing a New Planner Plugin"[7] Is now described the 2 phases of porting the code, the first one is the Nav2 structure porting procedure and the second part adapt the old code to ROS 2.

Porting code to ROS 1

In the package.xml that describe the package as an organizational unit for your ROS 2 code, the changes that are performed are:

- buildtool_depend : catking → ament_cmake : ament as its build system, there are 2 one for CMake based packages and mostly cpp ament_cmake and one for python ament_python

- roscpp → rclcpp: the new structure on how the client library are build are that at the fondation there is the general ros client library rcl writen in C and build on top of that are the rclcpp and rclpy, but are avaible in community maintened more.

- update the naming of all the packages

- to tell colcon the package is an ament_cmake add it in the build_type

Because we are working with cpp we need to edit also the CMakeLists.txt and the major changes are:

- update all the names of the packages

- set the new standard of C++ to 17

[4] https://wiki.ros.org/move_base

[5] https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html

[6] https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

[7] https://docs.nav2.org/plugin_tutorials/docs/writing_new_nav2planner_plugin.html

- `find_package()` required to be called for each package
- call `ament_export_dependencies()` to indicates which ament packages this package depends on at runtime
- write `ament_package()` at the end

Also in the code this was changes:

- the log calls are changed from `ROS_` to `RCLCPP_`
- the structure of some messages are changed
- add good practice to use share pointer

other changes are reported in the next section because are also connected with Nav2

Porting code to Nav 2

- in plugin definition: we move from thr base_class_type of nav_core::BaseGlobalPlanner to nav2_core::GlobalPlanner.
- because it is a plugin based on `nav2_core::GlobalPlanner` it inherits 5 pure virtual methods (`configure()`,`activate()`,`deactivate()`,`cleanup()`,`createPlan()`) that required override and used to compute the trajectories. Now we describe each method
  - `configure()` : called when node enters `on_configure` state, used for perform declarations of ROS parameters and initialization of planner's member variables. The input parameters are: shared pointer to parent node, planner name, tf buffer pointer and shared pointer to costmap. This replace the `initialize` method used in ROS 1 and also the class initialization. Also it's parametrs are used to link the planner information locally:
    * `rclcpp_lifecycle::LifecycleNode::WeakPtr` to link to the base class
    * the name of the plugin
    * a link to the `tf` with `std::shared_ptr<tf2_ros::Buffer>`
    * the map with `std::shared_ptr<nav2_costmap_2d::Costmap2DROS>`
  - `activate()` : called when node enters `on_activate` state, used for implement operations which are necessary before planner goes to an active state.
  - `deactivate()` : called when node enters `on_deactivate` state, used for implement operations which are necessary before planner goes to an inactive state.
  - `cleanup()` : called when node enters `on_cleanup` state, should clean up resources which are created for the planner.

- `createPlan()` : called when planner server demands a global plan for specified start and goal pose. replace the makePlan used in ROS 1 and return a path as `nav_msgs::msg::Path` using the start and end position as `geometry_msgs::msg::PoseStamped`

- at the end we placed the code that is used also or plugins:
```
#include "pluginlib/class_list_macros.hpp"
PLUGINLIB_EXPORT_CLASS(<type of the plugin class>, <type of the base class>);
```

- because it's a plugin we need to add a plugin declaration where is package.xml is located and edit the CMake file as described in the doc for plugins[8].

build the project

At the end we build the system using the new tool for building `colcon` and we have the global planner and also all the system ready to navigate through the facility.

[8]https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Pluginlib.html

Chapter 10

Hardware adaptation process from Rover Zero to Rover Mini

Non ho informazioni su nessun Rover Zero, rimuovo il capitolo?

Chapter 11

Integration of the navigation stack with Open-RMF

Now we discuss the steps taken to insert the rover Mini to the The Open Robotics Middleware Framework (Open-RMF), all this steps are taken following the documentation[1]. The first thing that we need to build is **route maps**, this maps are used to predict the navigation paths that the robots will took allowing RMF to avoid conflicts, this procedure is called "traffic management".

11.1 route maps generation

For this Open-RMF give the **traffic editor tool**[2] that allow to generate the route map. We started with the map obtained from Cartographer and start a new project `*.building.yaml`. Then, there are some drawing of the facility's plans and also that was added. This make us discover that the map acquired using Cartographer was not perfect and have a rotation error at one of the turning point of the building, visible in figure. We then to design the traffic flow for the floor following the best practice to obtain a optimize plan [3], the traffic editor allow to define this following components to the map that is going to be used as route map:

- **vertex**: they are waypoints or the definition of wall, door or else. The properties of a vertex can be:
  - `is_holding_point` : the robot is allowed to wait at this waypoint for an indefinite period of time.
  - `is_parking_spot`: robot is required to park itself.
  - `is_passthrough_point`: waypoint which the robot shouldn't stop.
  - `is_charger`: charging station for a robot
  - `is_cleaning_zone`: used for the `Clean` task, indicate if is a clean zone where to start the cleaning
  - `dock_name`: used when the robot is executing their custom docking sequence. send a command to rmf to dock the robot.

[1]https://osrf.github.io/ros2multirobotbook/intro.html
[2]https://github.com/open-rmf/rmf_traffic_editor
[3]https://osrf.github.io/ros2multirobotbook/integration_nav-maps-strategies.html

- pickup_dispenser: dispenser workcell for Delivery Task
- dropoff_ingestor: ingestor workcell for Delivery Task
- spawn_robot_type: used in simulation to spawn a robot model
- spawn_robot_name: used in simulation to identify a robot spawned

- measurement allow to add a measurement to set the scale of the drawing. In our case we used the know width of the hallway.
- door: used both for interacting with the real world than for simulation. if a robot is going to pass throught a lane where is a door, rmf will request to open the door.
- traffic lane: connect waypoints and are the paths for a fleet to follow, they are divided by groups and multiple fleet can be assign on a group. rmf suppose that are straight lines for control the vehicles. the lanes can be unidirectional or bidirectional and orientation if required the robot to navigate forward or backward.
- fiducials: used for align multiple floor and used as a link to ROS 2 map, are visible throught multiple floor and give an information about their position respect the map.
- lift: used for align multiple floor and used as a link to ROS 2 map, are visible throught multiple floor and give an information about their position respect the map.
- walls: for simulation add the walls to the world that will generate.
- floor: for simulation add the ground plane for robots wo walk on. can added hole for lifts.
- environment assets: for simulation add models of different furniture for gazebo.

This plan take into account for planning this factor:

- in the hallways can pass only one robot
- Holding points (leaf nodes) are added along the corridor or near the begin or the end of it for giving way to one another. If there are some task waypoint is suggested to add a very low limit speed to avoid to use them as holding points.
- mutex lanes or waypoints, At any point of time, only one robot is allowed to occupy any waypoint or lane belonging to the same mutex group, allow to avoids deadlocks.

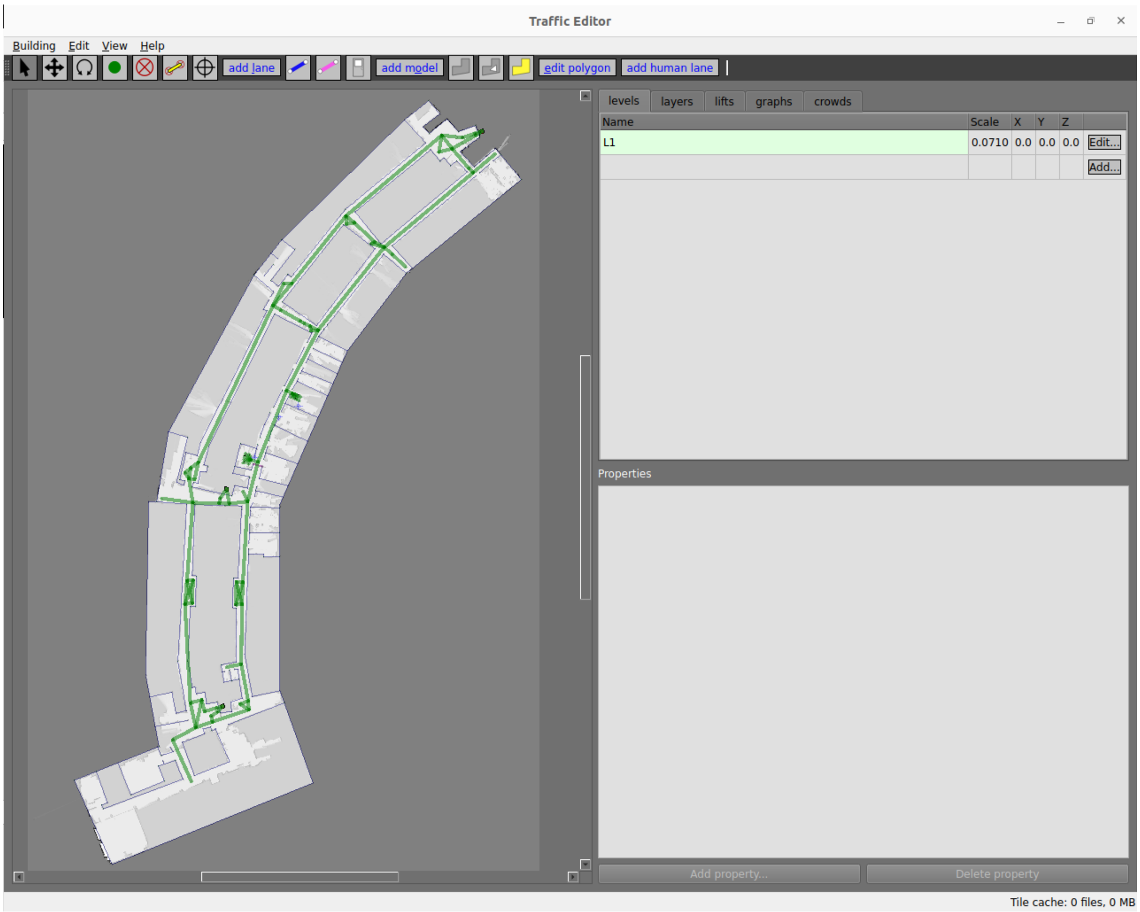

Figure 11.1: Traffic editor with Elettra's route map

Then it is possible to generate each route maps from each graphs `*.yaml` using the command:
`ros2 run rmf_building_map_tools building_map_generator nav \`
`<map.building.yaml> <folder/where/to/generate/routes>` Create the map we need a method to connect the rover to rmf, this is done through the **free fleet**

11.2 free fleet

Free fleet[4] is a python implementation of the `full_control` Open-RMF Fleet Adapter. It uses `zenoh` as a communication layer between each robot and the fleet adapter, allowing access and control over the navigation stacks of the robots.

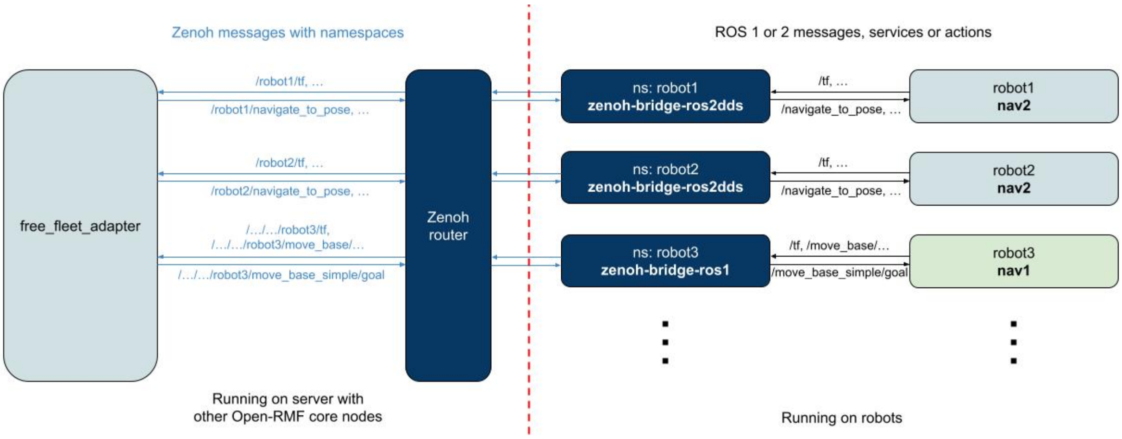

Figure 11.2: Free Fleet Adapter with Zenoh

The we edit the fleet configuration file to respect the characteristic of the fleet: name of the fleet, the limits in velocity, their size and mechanical info, their battery management, what task they are capable of and a list of all robot that are inside this fleet with its name, what charger use, frame used to calculate his position and information on the map. At the end we set the reference cordinate respect the robot frame and the rmf map.

11.2.1 Zenoh

Zenoh[5] is a pub/sub/query protocol unifying data in motion, data at rest and computations. It blends traditional pub/sub with geo distributed storage, queries and computations, while retaining a well level of time and space efficiency. It allow to send only specific topic through the web reducing bandwidth to a specific server. On this server is need to launch a Zenoh router with (`$ zenohd`), but there can be multiple instance of Zenoh router for redundancy. Then as described in the repo is suggested to use the binaries of `zenoh-bridge-ros2dds` one on the server and one on each robot with a configuration file. On the server it's configurated to accept all the traffic test se necessario avere avere il server from each machine can reach it, while on the client we need to setup:

- the `namespace` as the one associated to the file of configuration of the fleet
- allow to publish the frame status of the robot (`.*/tf` and `.*/tf_static`) and if we could also the battery information (`.*/battery_state`) and allow also the action servers that control the navigation(`.*/navigate_to_pose`)
- setup zenoh configuration defining the mode and the address of the server, with is protocol and its port that th client will `connect` and `listen` to (`tcp/<ip address server>:7447`)

[4]https://github.com/open-rmf/free_fleet
[5]https://zenoh.io/

11.3 launch RMF core

Setup the robot we can now launch the core of RMF using the launch file:
`0rmf_mini_world_rmf_core.launch.xml`
that launch:

- rmf_traffic_schedule : Traffic Schedule
- rmf_traffic_blockade : the traffic Blockade Moderator
- building_map_server : the build map
- visualization.launch.xml: that launch all the tool to see in rviz2 provare a rimuovere
- door_supervisor
- lift_supervisor
- rmf_task_dispatcher
- queue_manager: if request Enable the reservation node for parking spot deconfliction

11.4 web interface

launch with docker compose the we server and web interface and then launch: `ros2 launch myCode 1rmf_mini_fleet_adapter.launch.xml \`
`server_uri:="ws://localhost:8000/_internal"`

Chapter 12

Testing and evaluation of the system

The testing have brigh up some problem in the design of the rover. If needed to move in office the robot with the laser sensor cannot see part of the obstacle, see image:

Figure 12.1: Cartographer Algorithm

can be increase the occupancy map bigger but the dimension of the legs chair wouldn't allow to pass through port. A solution is to move the laser to a lower position but this required a low profile laser, that need some filtering to remove the wheel and obtaining blind spot without adding the sensor on top or adding another sensor like a deep camera that give a 3D occupancy greed that can be processed to

obtain a simple map.
The RMF commands from waypoint to waypoint: RMF commands the robot from waypoint to waypoint, and it leaves the low level navigation to each robot's navigation stack. So if the robot chooses to use a non-straight route, it means that the RMF navigation graph might need to be re-drawn or reconsidered (if there are constant obstacles around), or the navigation stack of the robot should be tuned to allow tighter tolerances for static/dynamic obstacles. but how the robot chooses to navigate between them is out of RMF's control, since RMF does not have dynamic obstacle information, or if there are new static obstacles, it needs to be reflected into RMF's traffic navigation graph

Part IV

Results and Analysis

Chapter 13

Presentation of results from testing and evaluation

Chapter 14

Comparison of system performance before and after the porting process

Chapter 15

Discussion on the challenges encountered and how they were addressed

Part V

Conclusion and Future Work

Part VI

Appendices

Chapter 16

Simulation

In this part i will write some useful information for running the simulation. All test were done on debian based machine: To see the graphical interface launched in the docker container you need to have to share the access to the X server using the command `xhost local:root`.

today the tool for converting the data are https://www.ros.org/blog/noetic-eol/

implement also planning benchmark

describe and use composition in nav2 to obtain better performance [7]

Inoltre per gestire il traffico sulla rete utilizzeremo zenoh (Zero Overhead Network Protocol) ed il bridge per ros2 zenoh-bridge-ros2dds che è pub/sub/query protocol unifying data in motion, data at rest and computations. It elegantly blends traditional pub/sub with geo distributed storage, queries and computations, while retaining a level of time and space efficiency that is well beyond any of the mainstream stacks. data liberator protocol. Zenoh liberates data in several dimensions.

with chatgpt possiamo gestire il robot in due modi: si ferma ed esegue l'azione non communicando con rmf (la linea rimane bloccata dal suo passaggio)

Bibliography

[1] Sy-Hung Bach, Phan-Bui Khoi, and Soo-Yeong Yi. "Application of QR Code for Localization and Navigation of Indoor Mobile Robot". In: *IEEE Access* 11 (2023), pp. 28384–28390. DOI: 10.1109/ACCESS.2023.3250253.

[2] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: http://dx.doi.org/10.1201/9780429489105.

[3] Tully Foote. "tf: The transform library". In: *Technologies for Practical Robot Applications (TePRA), 2013 IEEE International Conference on*. Open-Source Software workshop. 2013, pp. 1–6. DOI: 10.1109/TePRA.2013.6556373.

[4] Wolfgang Hess et al. "Real-Time Loop Closure in 2D LIDAR SLAM". In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278.

[5] Steve Macenski and Ivona Jambrecic. "SLAM Toolbox: SLAM for the dynamic world". In: *Journal of Open Source Software* 6.61 (2021), p. 2783. DOI: 10.21105/joss.02783. URL: https://doi.org/10.21105/joss.02783.

[6] Steve Macenski et al. *Impact of ROS 2 Node Composition in Robotic Systems*. 2023. arXiv: 2305.09933 [cs.RO]. URL: https://arxiv.org/abs/2305.09933.

[7] Steve Macenski et al. "Impact of ROS 2 Node Composition in Robotic Systems". In: *IEEE Robotics and Automation Letters* 8.7 (2023), pp. 3996–4003. DOI: 10.1109/LRA.2023.3279614.

[8] Steven Macenski et al. "Robot Operating System 2: Design, architecture, and uses in the wild". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

[9] Steven Macenski et al. "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.

[10] Alexey Merzlyakov and Steven Macenski. "A Comparison of Modern General-Purpose Visual SLAM Approaches". In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021.

[11] T. Moore and D. Stouch. "A Generalized Extended Kalman Filter Implementation for the Robot Operating System". In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.

[12] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2". In: *Robotics and Autonomous Systems* (2023).

[13] Kavan Singh Sikand et al. *Robofleet: Open Source Communication and Management for Fleets of Autonomous Robots*. 2021. arXiv: 2103.06993 [cs.RO]. URL: https://arxiv.org/abs/2103.06993.

[14] Victor Mayoral Vilches et al. *SROS2: Usable Cyber Security Tools for ROS 2*. 2022. arXiv: 2208.02615 [cs.CR]. URL: https://arxiv.org/abs/2208.02615.

[15] Andrey Vukolov. "D-Star-Based Optimized Trajectory Planner for Mobile Robots Operating in Dense Environments". In: *New Trends in Mechanism and Machine Science*. Ed. by Giulio Rosati, Alessandro Gasparetto, and Marco Ceccarelli. Cham: Springer Nature Switzerland, 2024, pp. 294–301. ISBN: 978-3-031-67295-8.

[16] Andrey Vukolov. *ElettraSciComp/witmotion_IMU_QT: Version 0.9.17*. Version v0.9.17-alpha. 2022. DOI: 10.5281/zenodo.7017118. URL: https://doi.org/10.5281/zenodo.7017118.

[17] Andrey Vukolov et al. "Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis". In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.